Comercial 360 — Historias de usuario y reparto del equipo

Oct 19, 2026 · @Luis Reyes Caro

Resumen del proyecto

Construimos **Comercial 360**, un gestor comercial (clientes, rutas, visitas, seguimientos, cotizaciones, pedidos y KPI), y lo sustentamos desplegado en AWS el Oct 26, 2026. Tenemos 6 días hábiles; cada persona es dueña de un módulo completo (backend + frontend) para poder explicarlo en la sustentación.

Requisitos obligatorios que no pueden faltar el día de la demo:

- [ ] Backend **NestJS** con arquitectura **Clean Architecture** (el enunciado acepta Hexagonal o Clean; el equipo eligió Clean)
- [ ] Persistencia con **TypeORM** sobre **PostgreSQL en Amazon RDS**
- [ ] Autenticación con **Better Auth**
- [ ] Mapa de visitas con **OpenStreetMap** (Leaflet)
- [ ] Dashboard de KPI, agenda del asesor, bitácora de actividades, cotizaciones simuladas, exportación **CSV**
- [ ] Todos los archivos (fotos, documentos) en **Amazon S3**; la BD guarda solo la referencia (key)
- [ ] App desplegada en **EC2** con **Jenkins** en la misma instancia, conectado a **GitHub**, y demo de un despliegue en vivo
- [ ] Repo sin archivos multimedia, credenciales ni `.env`

Los 5 diseños entregados (login, dashboard, registrar visita, creación de rutas) son la referencia visual del frontend. Roles que aparecen en los diseños: **Asesor Comercial** y **Director Regional**; añadimos **Administrador**.

Equipo y responsabilidades

El reparto es por **módulo vertical**: cada dev hace dominio, casos de uso, repositorios TypeORM, controlador, Swagger y pantallas de su módulo. Así nadie bloquea a nadie y cada uno sustenta lo suyo.

Persona	Rol	Módulo (épica)	Entidades que posee	Pantallas	Apoya a
Luis	Líder / arquitecto / DevOps	E0 · Infraestructura y arquitectura	Configuración TypeORM, módulo `shared`	—	Revisa todos los PR
Principal: Efrain	Backend + frontend	E1 · Autenticación, roles y asesores	Tablas de Better Auth, ASESOR	Login, layout (sidebar, header, filtros territorio/semana), asesores	Todos (guards y layout)
Principal: Yesid Colaboracion: Alejandro	Backend + frontend	E2 · Clientes, catálogos y bitácora	CLIENTE, CONTACTO_CLIENTE, ZONA, TIPO_CLIENTE, POTENCIAL, ESTADO_CLIENTE, PORTAFOLIO, CLIENTE_PORTAFOLIO, HISTORIAL_CLIENTE	Clientes, detalle de cliente, últimas interacciones	Alejandro y Camilo (datos de cliente)
Principal: Alejandro	Backend + frontend	E3 · Planeación de rutas	RUTA, RUTA_CLIENTE	Creación de rutas (wizard + mapa), rutas asignadas	Camilo (genera visitas)
Principal: Camilo	Backend + frontend	E4 · Agenda, visitas y seguimiento	VISITA, RESULTADO_VISITA, VISITA_RESULTADO, SEGUIMIENTO	Agenda, registrar visita	Valery (datos de KPI)
Principal: Carlos	Backend + frontend	E5 · Cotizaciones, pedidos y archivos S3	COTIZACION, COTIZACION_DETALLE, PEDIDO, referencias de archivos	Cotizaciones, pedidos, adjuntos	Todos (servicio de S3)
Principal: Valery	Backend + frontend + Jenkins	E6 · Dashboard y mapa de visitas	Consultas de solo lectura	Dashboard, mapa de visitas, tabla de seguimiento	Luis (configura Jenkins)
Principal: Valentina	Backend + frontend + QA	E7 · Reportes, indicadores y QA	Consultas de solo lectura para reportes	Reportes CSV, indicadores, widget de objetivo semanal	Todos (pruebas E2E y datos demo)

Responsabilidades del líder, además de E0:

- Daily de 15 min a las 9:00 y actualización de la columna Estado de las historias
- Merge a `develop` y `main`; nadie hace push directo a `main`

- Reporte de avance al tutor
- Dueño de las credenciales de AWS; las comparte por canal privado, nunca por el repo

Historias de usuario

42 historias en 8 épicas: 31 de prioridad Alta (mínimo para ganar), 8 Media y 3 Baja. Las Alta se cierran antes del viernes 23; Media y Baja solo si sobra tiempo. Cada dev actualiza la columna Estado de sus historias.

E0 · Infraestructura y arquitectura — Luis (Jenkins: Valery)

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-01	Como equipo, quiero un repo con NestJS en Clean Architecture y el frontend base, para trabajar en paralelo	Carpetas `domain / application / infrastructure` / presentation por módulo; TypeORM con migraciones (sin `synchronize`); `.env.example`; README con cómo correrlo	Alta	Luis	Pendiente
HU-02	Como equipo, quiero la BD en RDS PostgreSQL y un bucket S3, para cumplir la infraestructura exigida	RDS accesible solo desde la EC2; bucket privado; acceso por IAM role o variables de entorno; nada de llaves en el repo	Alta	Luis	Pendiente
HU-03	Como equipo, quiero la app desplegada en la EC2, para demostrarla funcionando en AWS	Backend NestJS corriendo con PM2 (o Docker); Nginx sirve el front y hace proxy a `/api`; `.env` solo en el servidor; migraciones corren contra RDS; script `deploy.sh` en el repo (pull → install → build → migraciones → reinicio) que Jenkins ejecuta en su etapa de deploy	Alta	Luis	Pendiente

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-41	Como equipo, quiero Jenkins configurado en la EC2 y conectado a GitHub, para que cada push a `main` dispare el despliegue	Jenkins instalado en la misma EC2 con usuario admin y puerto restringido; credenciales de GitHub y AWS guardadas en el gestor de credenciales de Jenkins; webhook de GitHub dispara el job al hacer push a `main`; `Jenkinsfile` en el repo con etapas checkout → install → build → deploy, donde deploy ejecuta el `deploy.sh` de Luis	Alta	Valery	Pendiente
HU-04	Como dev, quiero estándares comunes (validación, errores, Swagger, paginación), para que todos los módulos respondan igual	`ValidationPipe` global; filtro de excepciones; Swagger en `/api/docs`; DTO de paginación compartido	Media	Luis	Pendiente

E1 · Autenticación, roles y asesores — Efrain

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-05	Como usuario, quiero iniciar sesión con correo y contraseña, para acceder a la plataforma	Better Auth en NestJS; login, logout y "Recordarme"; rutas del front protegidas; guard de sesión en el back	Alta	Efrain	Pendiente
HU-06	Como administrador, quiero asignar roles (Admin, Director Regional, Asesor), para restringir lo que cada uno ve	Decorador `@Roles`; el asesor solo ve sus rutas, visitas y cotizaciones; el director ve su zona	Alta	Efrain	Pendiente
HU-07	Como administrador, quiero gestionar asesores, para asignarles rutas	CRUD de ASESOR ligado a un usuario; correo único; activar/inactivar (sin borrado físico)	Alta	Efrain	Pendiente

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-08	Como usuario, quiero un layout común (menú lateral, buscador, territorio, semana, perfil), para navegar como en el diseño	Sidebar con todos los módulos; filtros de territorio y semana disponibles para las demás pantallas; menú de perfil con cerrar sesión	Alta	Efrain	Pendiente
HU-09	Como usuario, quiero recuperar mi contraseña, para no quedar bloqueado	Enlace "¿Olvidaste tu contraseña?" funcional con Better Auth	Baja	Efrain	Pendiente
HU-10	Como usuario, quiero entrar con Microsoft 365, como muestra el diseño	Proveedor social Microsoft de Better Auth	Baja	Efrain	Pendiente

E2 · Clientes, catálogos y bitácora — Yesid

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-11	Como asesor, quiero registrar y editar clientes, para centralizar su información	Todos los campos de CLIENTE; NIT único; latitud y longitud (campo añadido al modelo, se obtiene con Nominatim de OSM) para ubicarlo en el mapa; inactivar en vez de borrar	Alta	Yesid	Pendiente
HU-12	Como asesor, quiero buscar y filtrar clientes, para encontrarlos rápido	Búsqueda por NIT, razón social o nombre; filtros por zona, tipo, municipio y potencial; paginado	Alta	Yesid	Pendiente
HU-13	Como asesor, quiero gestionar los contactos de un cliente, para saber con quién hablar	CRUD de CONTACTO_CLIENTE; solo un contacto principal por cliente	Alta	Yesid	Pendiente
HU-14	Como administrador, quiero administrar catálogos, para no tener valores fijos en el código	Zona, tipo de cliente, potencial, estado de cliente, resultado de visita y portafolio; seed inicial con datos de Barranquilla, Soledad, Malambo y Galapa	Alta	Yesid	Pendiente

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-15	Como sistema, quiero registrar en bitácora cada acción sobre un cliente, para tener trazabilidad	HISTORIAL_CLIENTE guarda fecha, usuario, acción y descripción; se expone la interfaz `ActivityLogService` que usan visitas, cotizaciones y pedidos; vista "Últimas interacciones"	Alta	Yesid	Pendiente
HU-16	Como asesor, quiero asociar portafolios a un cliente, para saber qué líneas le interesan	CLIENTE_PORTAFOLIO con fecha de registro; agregar y quitar desde el detalle	Media	Yesid	Pendiente

E3 · Planeación de rutas — Alejandro

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-17	Como director regional, quiero crear una ruta con sus datos generales, para planear la semana	Paso 1 del wizard: nombre, zona, día, frecuencia, horario, descripción, fechas; guardar como borrador	Alta	Alejandro	Pendiente
HU-18	Como director regional, quiero agregar clientes a la ruta y ordenar las paradas, para definir el recorrido	Buscar clientes por nombre, tipo y municipio; agregar/quitar; arrastrar para reordenar (`orden_visita`); hora sugerida por parada	Alta	Alejandro	Pendiente
HU-19	Como director regional, quiero ver la ruta en un mapa, para validar el recorrido	Leaflet + OpenStreetMap; paradas numeradas unidas por una línea; se actualiza al reordenar	Alta	Alejandro	Pendiente
HU-20	Como director regional, quiero asignar la ruta a un asesor, para que aparezca en su agenda	Paso 3: elegir asesor; al guardar se crean las VISITAS programadas según día y frecuencia (llama al caso de uso de Camilo)	Alta	Alejandro	Pendiente
HU-21	Como director regional, quiero ver las rutas asignadas, para controlar la operación	Listado con filtros por asesor, zona y estado; ver, editar e inactivar	Media	Alejandro	Pendiente

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-22	Como director regional, quiero ver distancia y tiempo estimado de la ruta	Resumen: número de paradas, km y tiempo (OSRM público o cálculo aproximado)	Baja	Alejandro	Pendiente

E4 · Agenda, visitas y seguimiento — Camilo

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-23	Como asesor, quiero ver mi agenda del día y de la semana, para saber a quién visitar	Visitas ordenadas por hora con estado (Pendiente, Visitado, Reprogramado, Sin visitar); detalle con contacto, objetivo y potencial; botón "Iniciar visita"	Alta	Camilo	Pendiente
HU-24	Como asesor, quiero registrar el resultado de una visita, para dejar constancia	Formulario del diseño "Registrar visita": fecha real, hora inicio/fin, resultado(s), observaciones (máx. 500), estado; escribe en la bitácora	Alta	Camilo	Pendiente
HU-25	Como asesor, quiero programar un seguimiento, para no perder la oportunidad	SEGUIMIENTO con próxima fecha, actividad, observación y responsable; marcar como completado; aparece en la agenda	Alta	Camilo	Pendiente
HU-26	Como asesor, quiero reprogramar o marcar una visita como no realizada	Cambia estado y fecha programada; queda en bitácora	Media	Camilo	Pendiente
HU-27	Como asesor, quiero indicar desde la visita si se generó cotización o pedido	Sección "Gestión comercial": si se marca Sí, abre la creación de cotización/pedido de Carlos con el cliente ya cargado	Media	Camilo	Pendiente
HU-28	Como asesor, quiero adjuntar una foto o documento a la visita	Usa el servicio de archivos S3 de Carlos; muestra miniatura o enlace	Alta	Camilo	Pendiente

E5 · Cotizaciones, pedidos y archivos S3 — Carlos

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-29	Como dev, quiero un servicio de archivos compartido, para que todo módulo suba a S3 igual	Interfaz `FileStorageService` en la capa de aplicación + implementación S3 en infraestructura; la BD guarda la key; descarga por URL firmada temporal; límite de tamaño y tipos permitidos	Alta	Carlos	Pendiente
HU-30	Como asesor, quiero crear una cotización con productos, para enviarla al cliente	Número consecutivo `COT-0001`; detalle con producto, cantidad y precio; subtotal y total calculados en el backend	Alta	Carlos	Pendiente
HU-31	Como asesor, quiero adjuntar el documento de la cotización, para tener soporte	Sube PDF/imagen a S3; se ve y descarga desde el detalle	Alta	Carlos	Pendiente
HU-32	Como asesor, quiero cambiar el estado de la cotización	Borrador → Enviada → Aprobada / Rechazada; cada cambio va a la bitácora	Alta	Carlos	Pendiente
HU-33	Como asesor, quiero convertir una cotización aprobada en pedido	Crea PEDIDO `PED-0001` con el valor de la cotización; estados del pedido (Creado, Despachado, Entregado, Anulado)	Alta	Carlos	Pendiente
HU-34	Como asesor, quiero listar cotizaciones y pedidos con filtros	Filtros por cliente, asesor, estado y fechas; paginado	Media	Carlos	Pendiente

E6 · Dashboard y mapa de visitas — Valery

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-35	Como director, quiero ver los KPI de la semana, para tomar decisiones	Tarjetas: visitas programadas, visitas realizadas, efectividad %, valor cotizado, pedidos cerrados, facturación; filtran por territorio y semana	Alta	Valery	Pendiente

ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-36	Como director, quiero un mapa de visitas, para ver la operación del día	Leaflet + OSM; marcadores por estado con la leyenda del diseño (Visitado, Pendiente, Sin visitar, Pedido cerrado, Reprogramado)	Alta	Valery	Pendiente
HU-37	Como director, quiero la tabla "Seguimiento de rutas y visitas"	Columnas del diseño; filtros y paginación en el backend	Alta	Valery	Pendiente

E7 · Reportes, indicadores y QA — Valentina

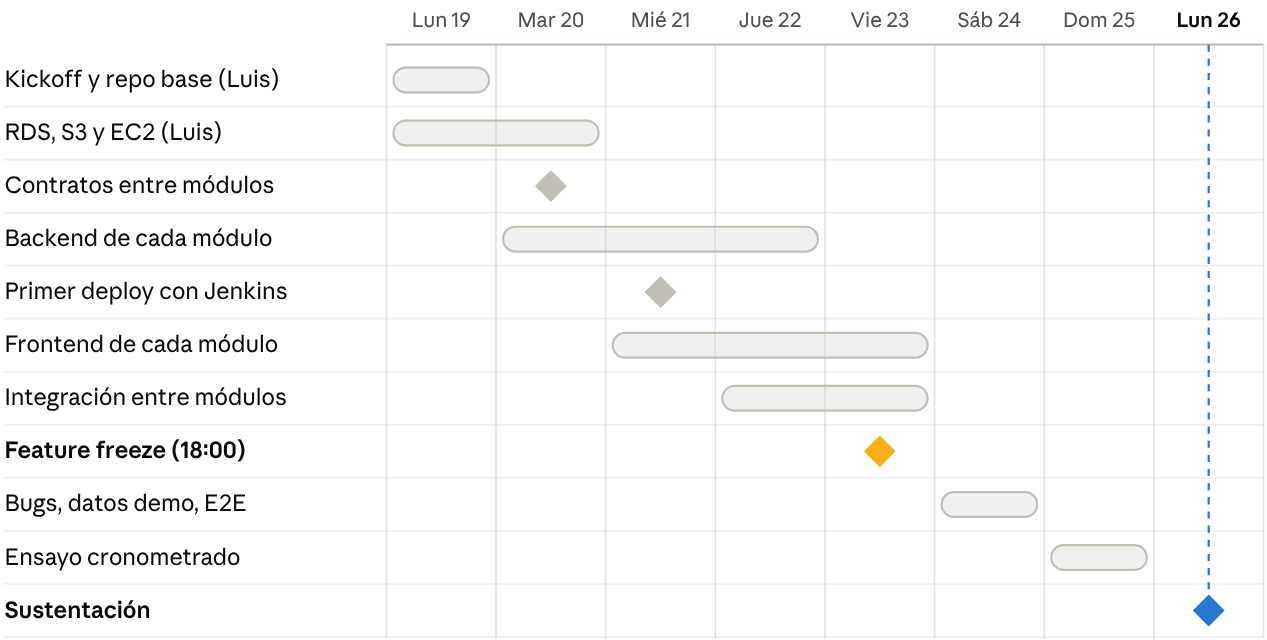
ID	Historia	Criterios de aceptación	Prioridad	Responsable	Estado
HU-38	Como director, quiero exportar reportes a CSV	Exporta visitas, clientes, cotizaciones y pedidos con los filtros activos; abre bien en Excel (UTF-8 con BOM)	Alta	Valentina	Pendiente
HU-39	Como asesor, quiero ver mi % de cumplimiento del objetivo semanal	Widget del sidebar: meta, completadas y barra de progreso	Media	Valentina	Pendiente
HU-40	Como director, quiero indicadores por asesor y por zona	Página "Indicadores" con gráficas de efectividad y valor cotizado	Media	Valentina	Pendiente
HU-42	Como equipo, quiero datos demo y pruebas de punta a punta, para llegar a la sustentación sin fallas	Seed con 3 asesores, 20 clientes de Barranquilla y alrededores y 4 rutas; checklist E2E del flujo login → ruta → visita → cotización → pedido → dashboard → CSV, ejecutado en AWS el sábado 24; cada fallo se reporta como issue en GitHub con su responsable	Alta	Valentina	Pendiente

Cronograma

Todo lo de prioridad Alta queda integrado el viernes 23 a las 18:00; sábado y domingo no se agregan funciones, solo se corrigen errores y se ensaya.

Funcionalidad congelada el viernes 23; el fin de semana solo se pule y ensaya

Plan del equipo por día, 19 al 26 de octubre de 2026



cronograma · 19 al 26 de octubre

Entregable de cada día (se revisa en la daily de las 9:00):

- [ ] **Lun 19:** repo y estructura de Clean Architecture creados (Luis); cada dev define entidades y endpoints de su módulo
- [ ] **Mar 20:** RDS, S3 y EC2 arriba; contratos acordados a las 12:00; login listo para que todos usen el guard (Efrain); seed de catálogos (Yesid)
- [ ] **Mié 21:** endpoints principales en Swagger; Jenkins configurado (Valery) y primer despliegue en EC2 (Luis)
- [ ] **Jue 22:** pantallas conectadas al backend; flujo ruta → agenda → visita funcionando
- [ ] **Vie 23:** flujo completo visita → cotización → pedido → dashboard; freeze a las 18:00
- [ ] **Sáb 24:** solo bugs, datos demo en RDS y prueba de punta a punta en AWS (coordina Valentina)
- [ ] **Dom 25:** ensayo cronometrado, prueba de deploy en vivo y video de respaldo
- [ ] **Lun 26:** sustentación, todo el equipo presente

Reglas de trabajo

Clean Architecture. Todos siguen la misma estructura por módulo, con 4 capas y las dependencias apuntando siempre hacia adentro; Luis la deja creada el lunes con el módulo de ejemplo `zones`.

```
src/
  shared/                     # pagination, exceptions, ActivityLogService,
FileStorageService (abstract classes)
  modules/
    clients/
      domain/                 # entities, value objects, repository
interfaces (abstract classes), domain errors
      application/            # use cases (CreateClientUseCase, ...),
input/output DTOs, service interfaces
      infrastructure/         # TypeORM entities + mappers, repository
implementations, migrations, external clients
      presentation/           # controllers, request/response DTOs, Swagger
decorators
      clients.module.ts       # binds each abstract class to its
implementation (DI)
```

- Regla de dependencias: `domain` no importa nada externo (ni NestJS ni TypeORM); `application` solo importa `domain`; `infrastructure` y `presentation` dependen de las capas internas, nunca al revés.
- Los casos de uso dependen de clases abstractas (ej. `ClientRepository`) y el `*.module.ts` las enlaza con su implementación TypeORM; las entidades de dominio no son las entidades de TypeORM, un mapper traduce entre ambas. Un módulo usa a otro solo por esas abstracciones (ej. visitas llama a `ActivityLogService`, nunca al repositorio de historial).
- Cada dev crea las migraciones de sus tablas; nombres de tablas y columnas en inglés y `snake_case`.
- Código, comentarios, Swagger y mensajes de error en inglés.

Git y GitHub.

- Ramas: `main` (lo que despliega Jenkins), `develop` (integración), `feature/HU-XX-descripcion-corta`.
- PR a `develop` con al menos 1 aprobación (Luis o el dev del módulo vecino); Luis pasa `develop` a `main` al final de cada día.

- Commits convencionales: `feat(clients): add NIT uniqueness validation (HU-11)`.
- Prohibido subir `.env`, llaves, imágenes o PDFs; `.gitignore` lo cubre desde el primer commit.

Definición de terminado (una historia pasa a Hecho solo si cumple todo):

- [ ] Cumple sus criterios de aceptación
- [ ] Endpoint documentado en Swagger y DTOs validados
- [ ] Migración incluida y corre en RDS
- [ ] Pantalla conectada al backend real (sin datos quemados)
- [ ] PR aprobado y mergeado a `develop`
- [ ] Probada en el entorno desplegado en EC2

Contratos entre módulos (acordar el martes antes de las 12:00):

Proveedor	Consumidor	Contrato
Efrain	Todos	`@Roles()`, `@CurrentUser()` y guard de sesión
Yesid	Camilo, Carlos	`ActivityLogService.register(clientId, action, description, user)`
Yesid	Alejandro, Valery	`GET /clients` con filtros y lat/lng
Camilo	Alejandro	`ScheduleVisitsForRouteUseCase(routeId)`
Carlos	Camilo	`FileStorageService.upload(file, folder)` → key; `getSignedUrl(key)`
Camilo, Carlos	Valery, Valentina	Campos de estado y valores para calcular KPI
Luis	Valery	deploy.sh en el repo, que la etapa de deploy del Jenkinsfile ejecuta

Plan de sustentación (60 min)

La demo sigue el flujo real de un asesor, así cada persona presenta su módulo en orden y se ve la integración de punta a punta. Se evalúa que **todos** puedan explicar su aporte.

Minuto	Quién	Qué muestra	
0–5	Luis	Problema, solución y roles del sistema	
5–12	Luis	Clean Architecture (capas y regla de dependencias, con un módulo de ejemplo) y diagrama AWS: EC2, RDS, S3, Jenkins	
12–18	Efrain	Login con Better Auth, roles, asesores y cómo funciona el guard	
18–24	Yesid	Crear cliente con contactos, catálogos y bitácora automática	
24–30	Alejandro	Crear ruta, ordenar paradas en el mapa y asignar a un asesor	
30–36	Camilo	Agenda del asesor, registrar visita con foto y programar seguimiento	
36–42	Carlos	Cotización con documento en S3 (mostrar la key en BD y el objeto en el bucket) y conversión a pedido	
42–46	Valery	Dashboard con los KPI ya actualizados por la demo y mapa de visitas	
46–50	Valentina	Exportación CSV, indicadores y cómo se probó el flujo de punta a punta	
50–55	Luis	Cambio pequeño en vivo: push a GitHub → Jenkins despliega → se ve en la app	
55–60	Todos	Preguntas	

Preparación:

- [ ] Datos demo cargados en RDS (3 asesores, 20 clientes reales de Barranquilla y alrededores, 4 rutas)
- [ ] Usuarios de prueba por rol con contraseñas anotadas fuera del repo
- [ ] Ensayo completo cronometrado el domingo 25
- [ ] Plan B: video grabado de la demo por si falla la red